Amazon RDS PostgreSQL Database Logging

To Begin with the Lab

Summary of the Lab

In this lab, PostgreSQL database logging was explored using Amazon RDS. SQL statements were executed from a PostgreSQL client session running on a Bastion Host, and the generated logs were viewed directly from the RDS console. The session-level logging parameter was modified to capture all SQL statements, allowing queries and errors to appear in the database logs. The lab demonstrated how PostgreSQL logging works, how logs can be viewed in real time from the RDS console, and how database administrators can use these logs for troubleshooting, auditing, and performance analysis.

Understanding PostgreSQL Database Logging in Amazon RDS

PostgreSQL logging records database activity such as SQL queries, user connections, warnings, and errors. These logs help database administrators understand what is happening inside the database and troubleshoot issues quickly. By controlling logging parameters, administrators can decide whether to log all statements, only errors, or specific events. This provides visibility into database operations without requiring additional monitoring tools.

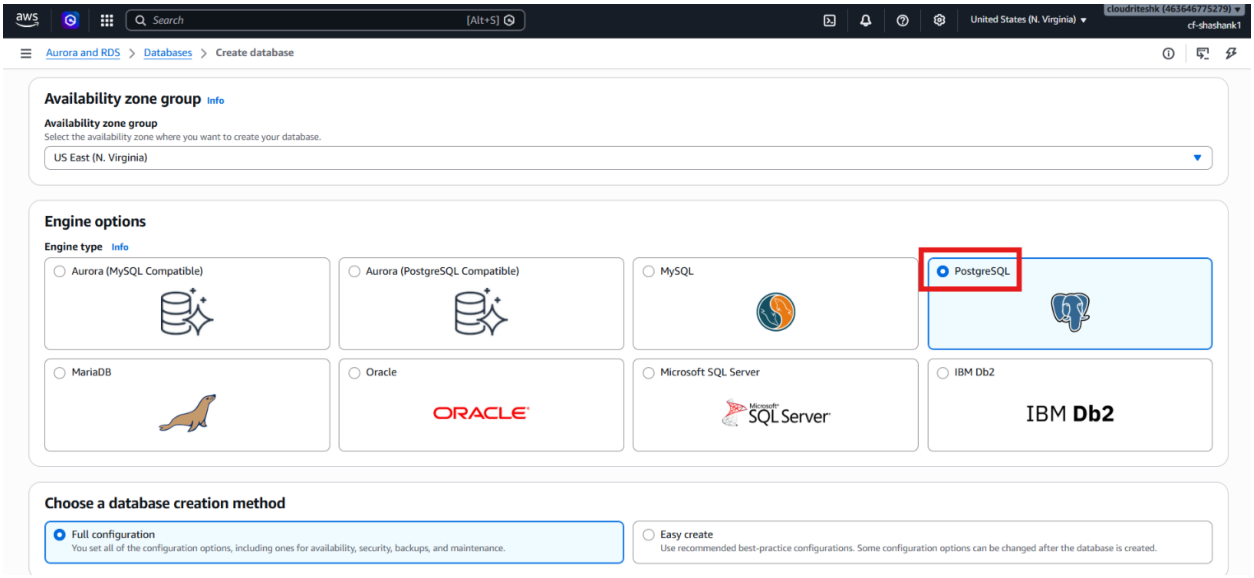
Example:

A developer reports that an application query is failing. By enabling statement logging and reviewing the database logs, the administrator can identify the exact SQL query being executed and any associated error messages. This makes it easier to diagnose the problem and implement a fix.

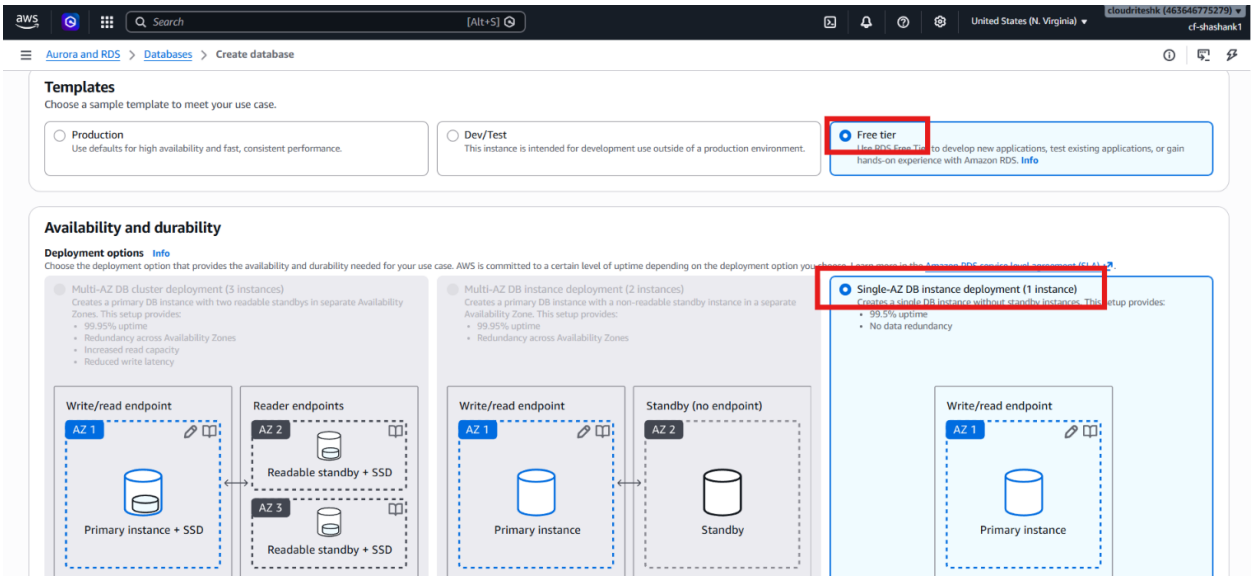
Lab Steps

Step 1 – Create any Database with monitoring enabled

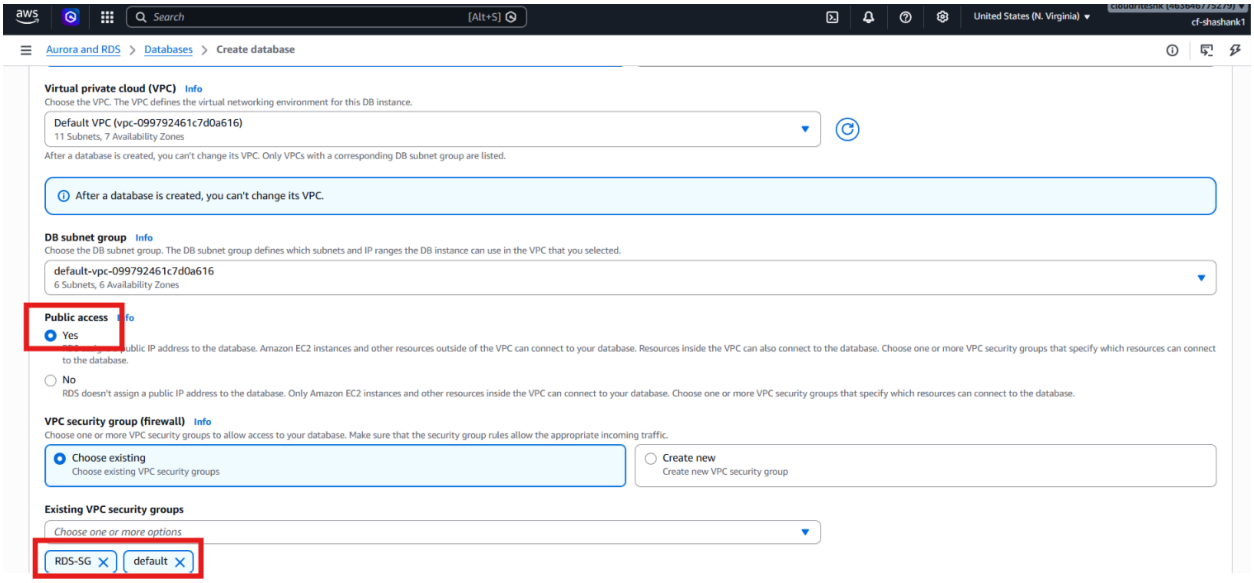
- I will create the Free Tier Database for lab purpose.
- Select **Aurora PostgreSQL** or **Amazon RDS PostgreSQL** based on the lab setup.



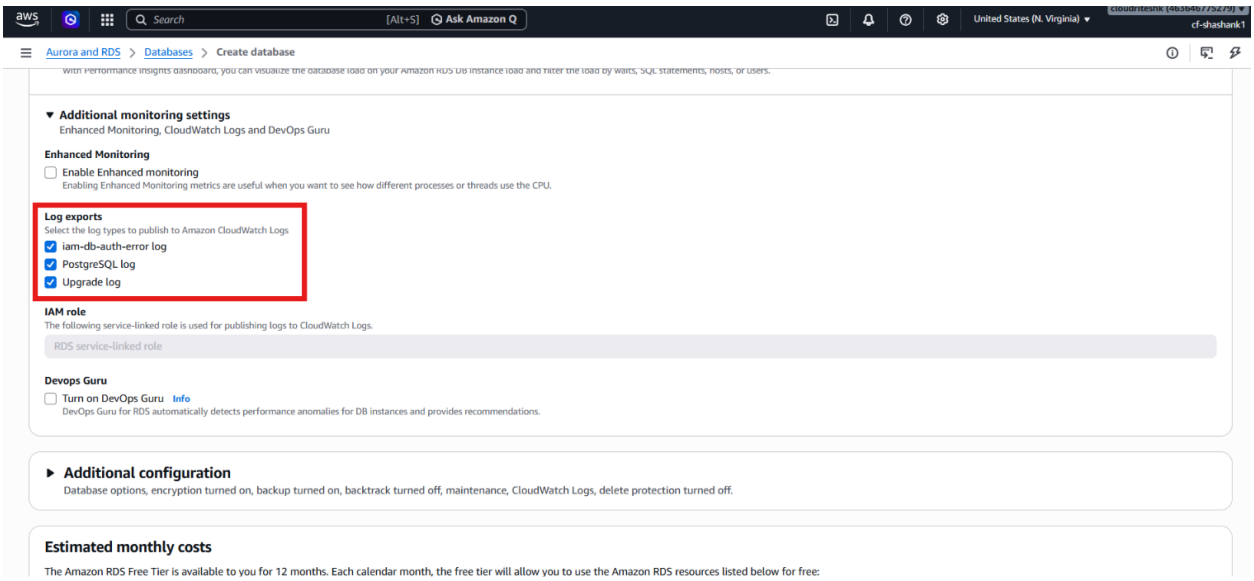
- Choose the current production version.



- Set the database identifier.
- Configure the master username and password.
- Attach the required security group.
- Set **Public Access** to **Yes**.



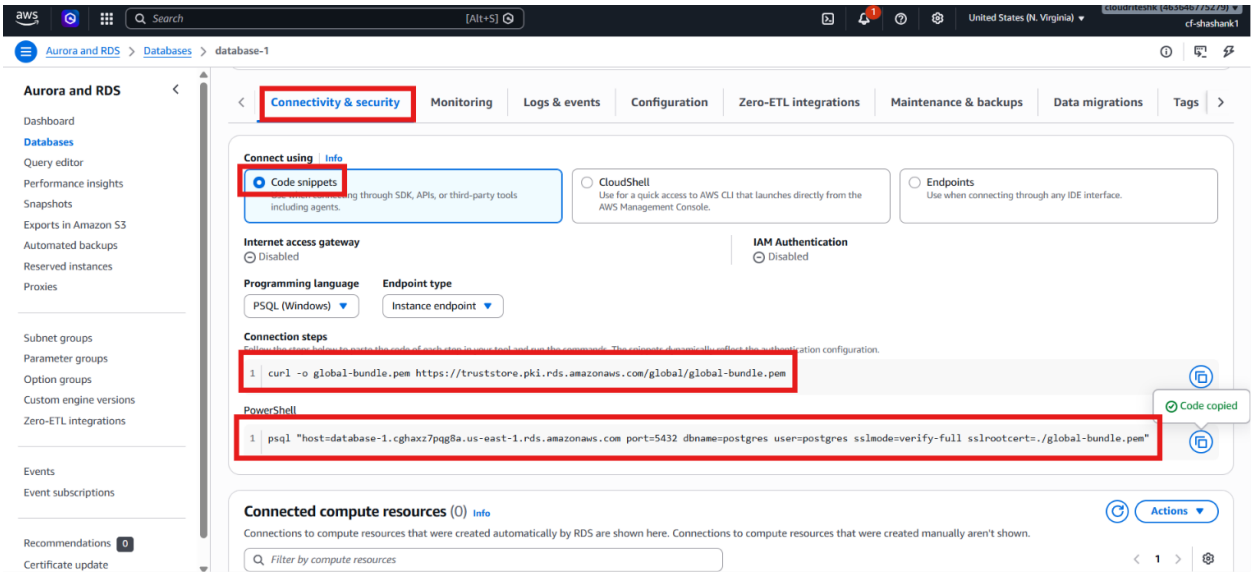
- Tick all three but we will be watching mainly **PostgreSQL log**.



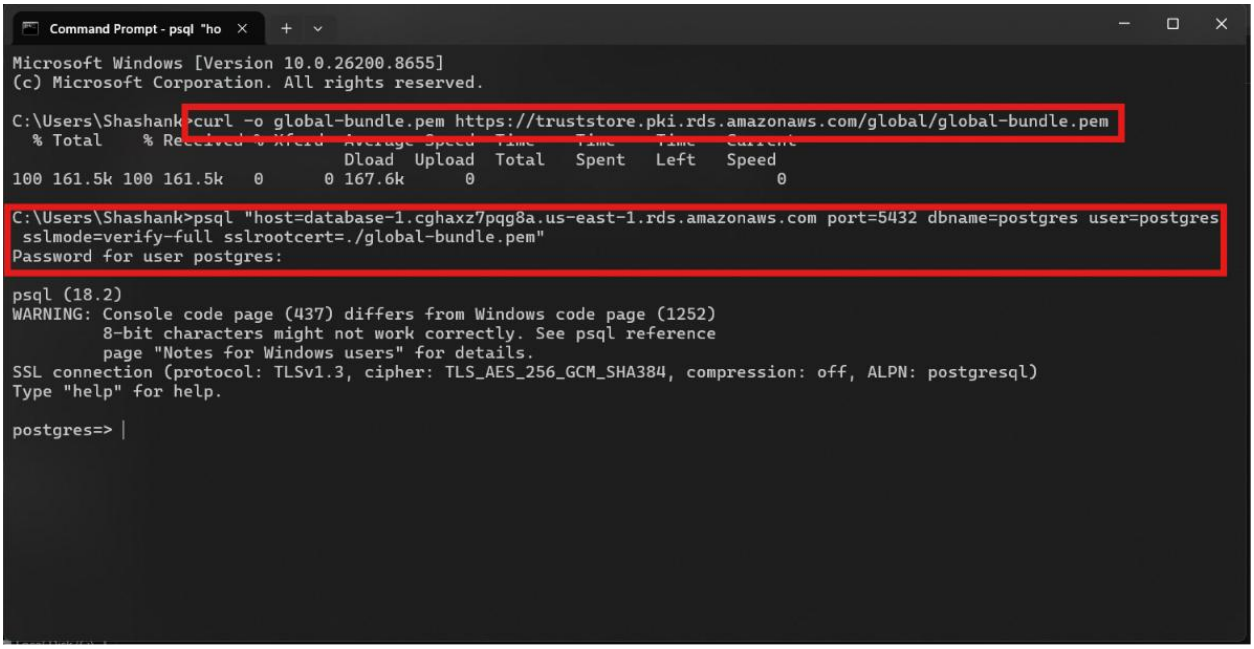
- Create the database and wait for it to become available.

Step 2 – Connect to the Database

- On the **Connectivity & security** panel connect using Code Snippets and run the commands in Command Prompt to connect to database.

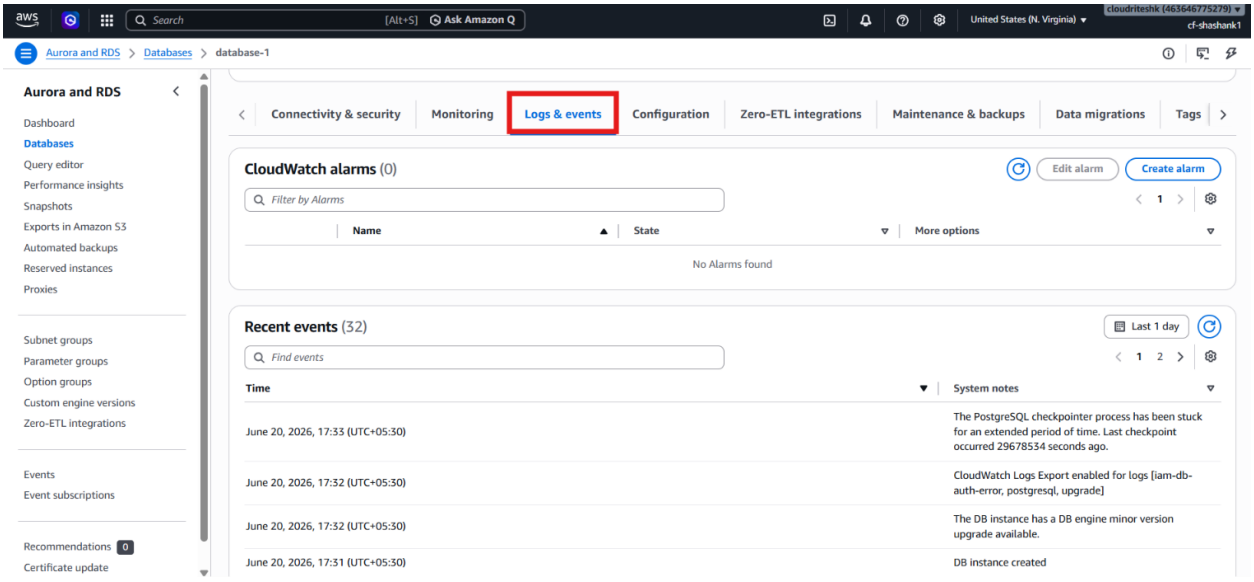


- Enter your password and you are connected to database.

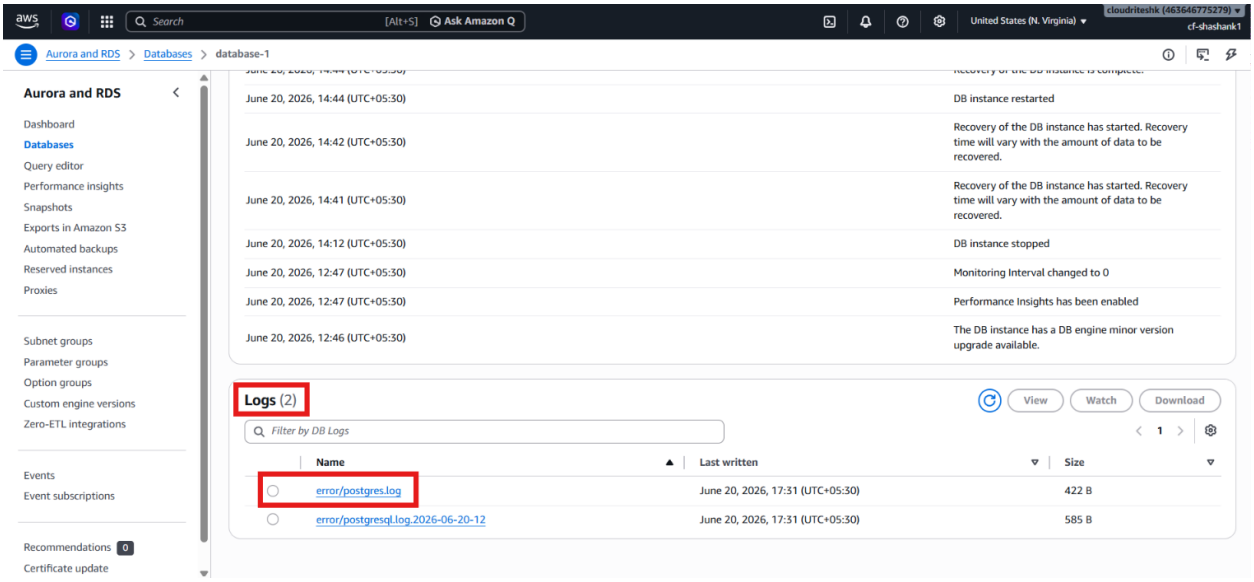


Step 3 – Watch Logs and Create tables

- Open your database and go to **Logs & Events** and scroll down to **Logs**.



- To view any log, click on it.



- After connecting to database, we will run some statements but none of them will be logged, so we must set log_statement to all to let it log all statements from now onwards.
- For this, we will run the following command and run statements and watch logs to see if anything changed on log.
- NOTE – The log statement will be valid for this session only, if you do disconnect it, you have to run this command again to log the statements in future.

```
SET log_statement='all';
```

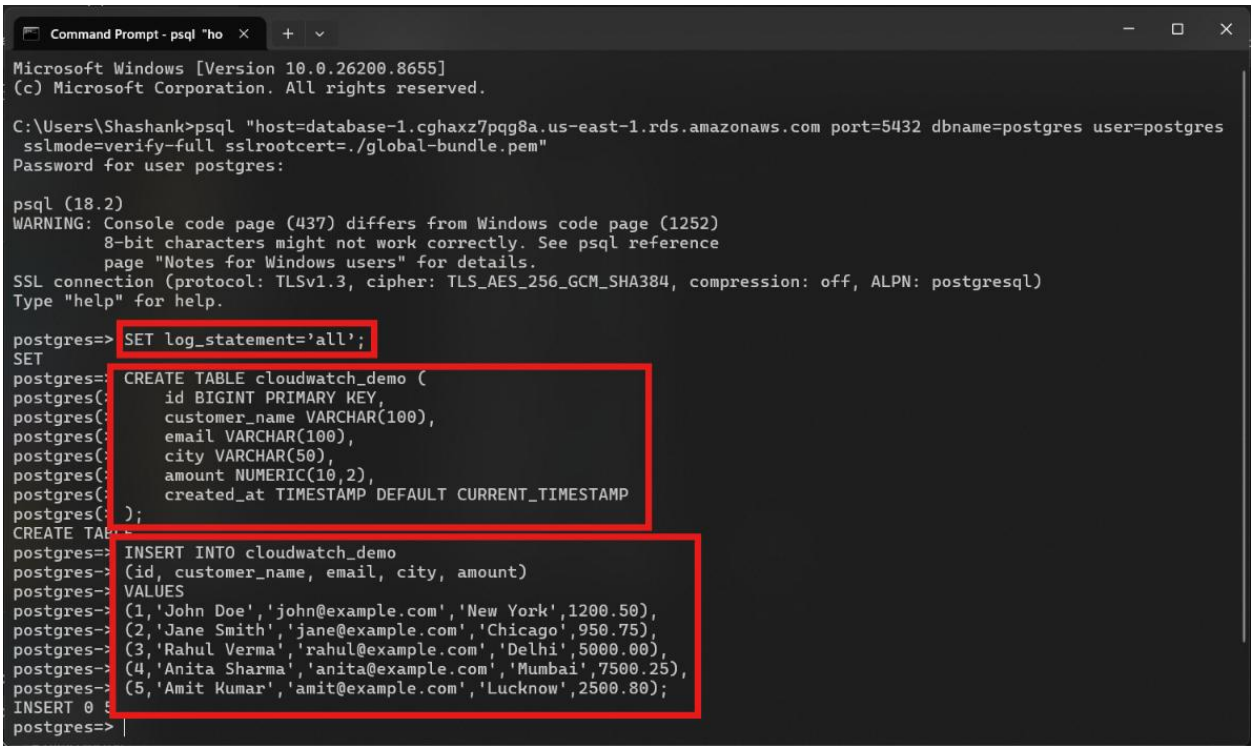
- Create tables and insert the values for lab purpose.

```
CREATE TABLE cloudwatch_demo (
    id BIGINT PRIMARY KEY,
    customer_name VARCHAR(100),
```

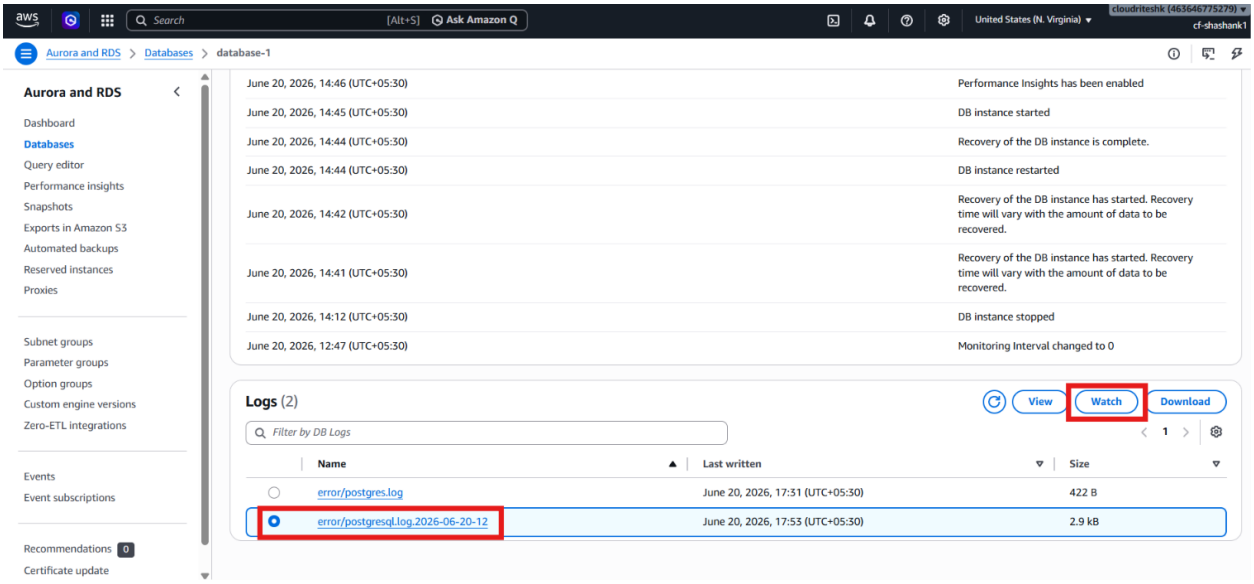
```
    email VARCHAR(100),
    city VARCHAR(50),
    amount NUMERIC(10,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

- Insert data

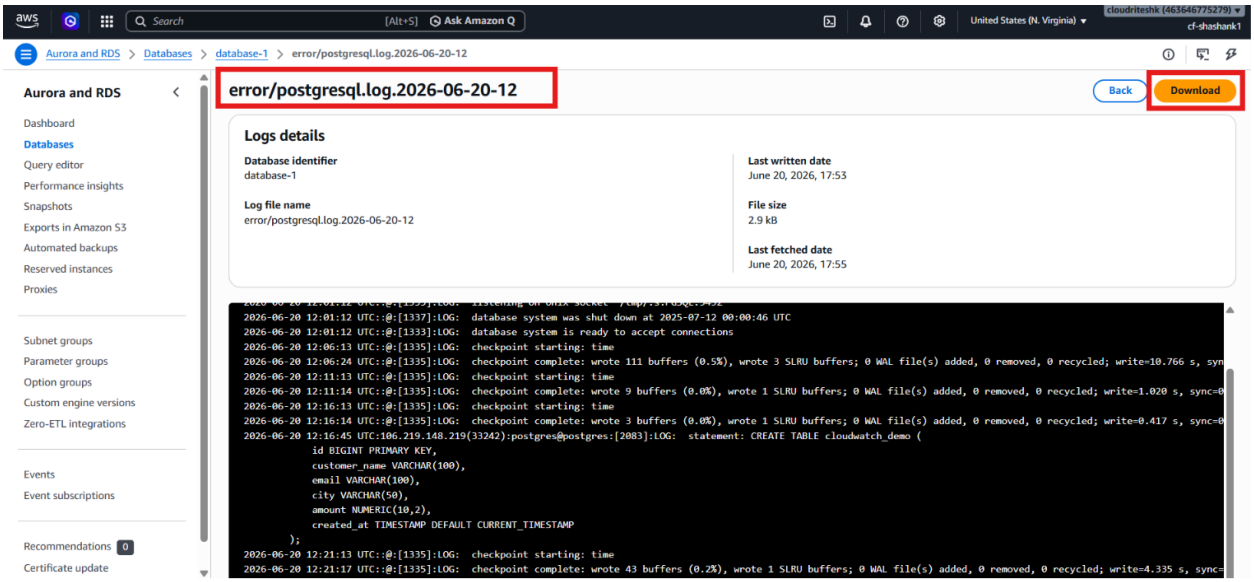
```
INSERT INTO cloudwatch_demo
(id, customer_name, email, city, amount)
VALUES
(1,'John Doe','john@example.com','New York',1200.50),
(2,'Jane Smith','jane@example.com','Chicago',950.75),
(3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
(4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
(5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
```



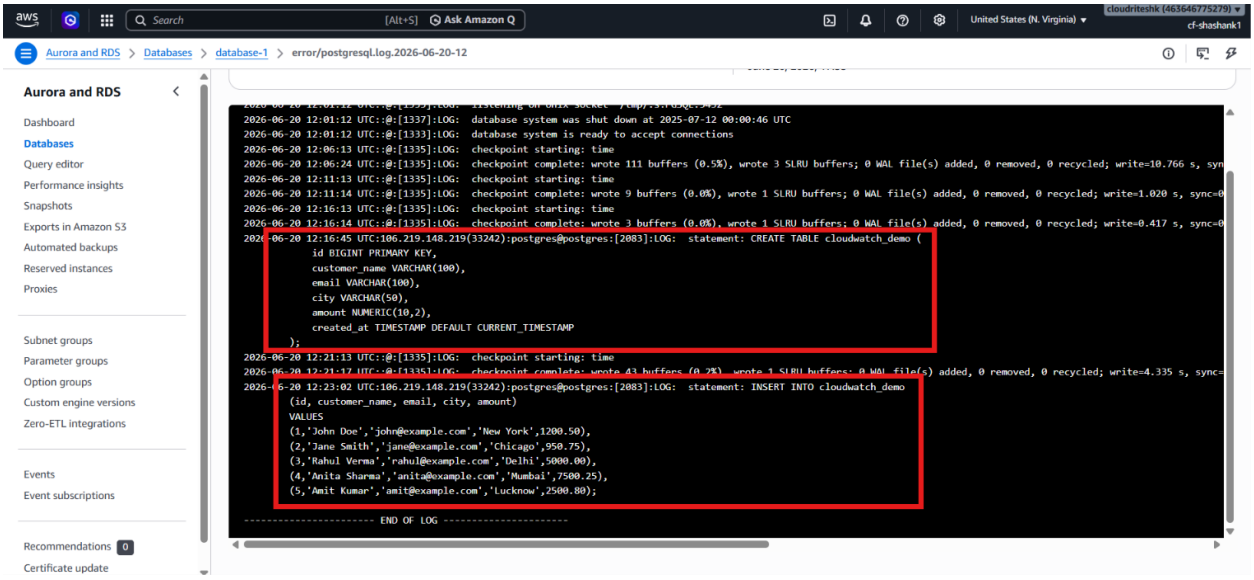
- Now go to logs, select it and click on Watch.



- Now we can see the log being updated automatically and we can also download it from here.



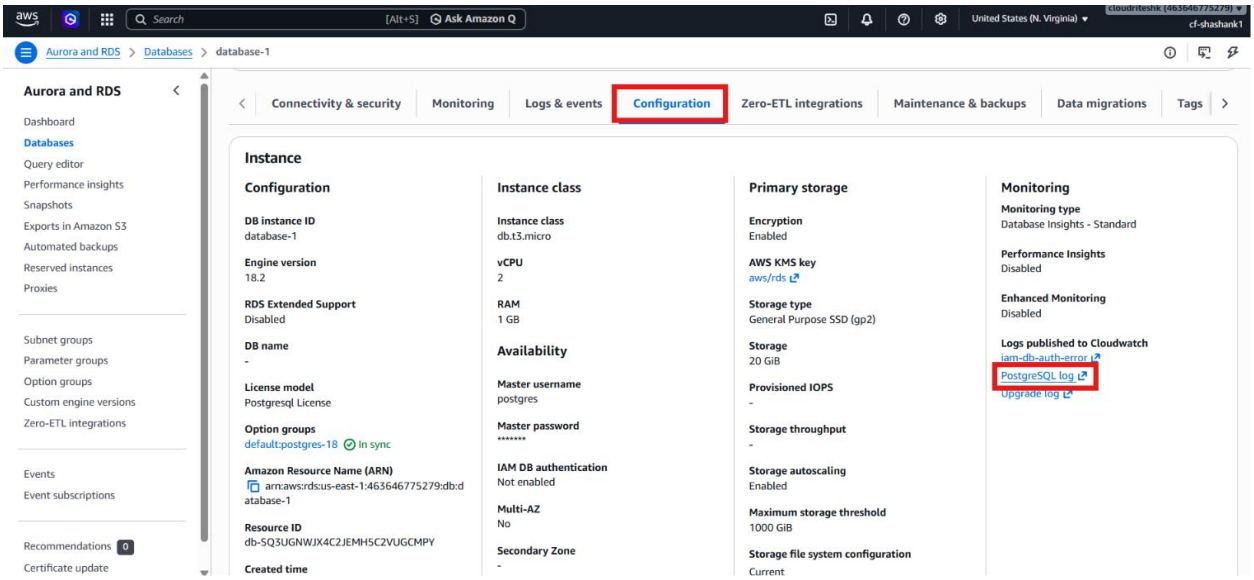
- Here we can see our statements getting logged as we ran them earlier.



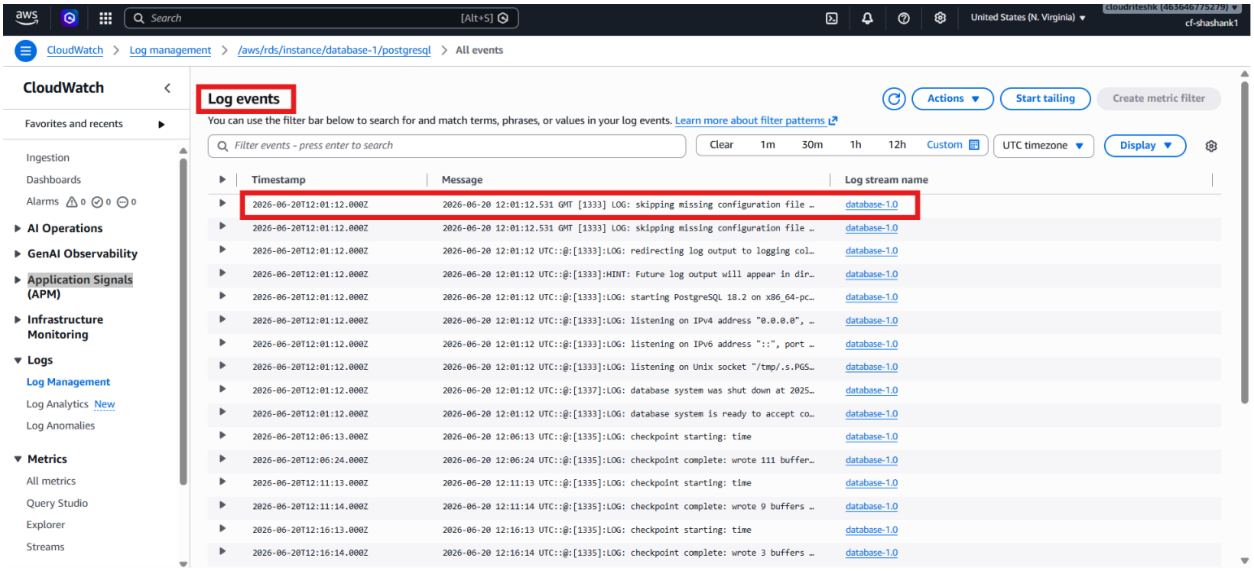
- Here

Step 4 – Watch Logs in CloudWatch

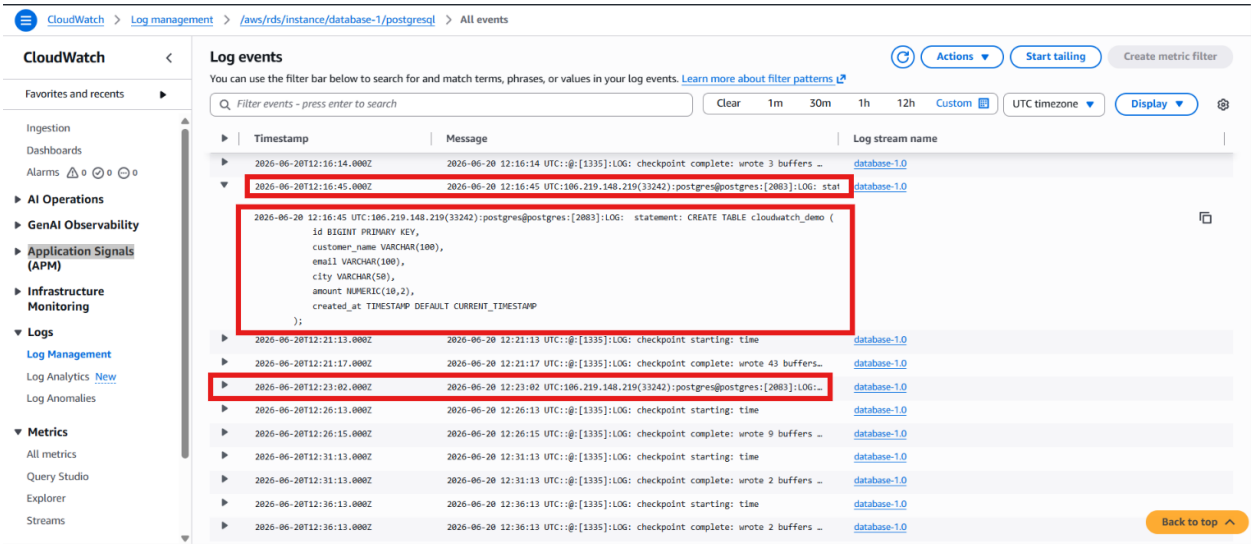
- Earlier while creating the database, we checked the tickbox **PostgreSQL logs** for exporting logs to CloudWatch, so our logs are already being exported to CloudWatch.
- Now we will go to CloudWatch and see if our logs appeared there.
- Open the Configuration tab and click on **PostgreSQL log**.



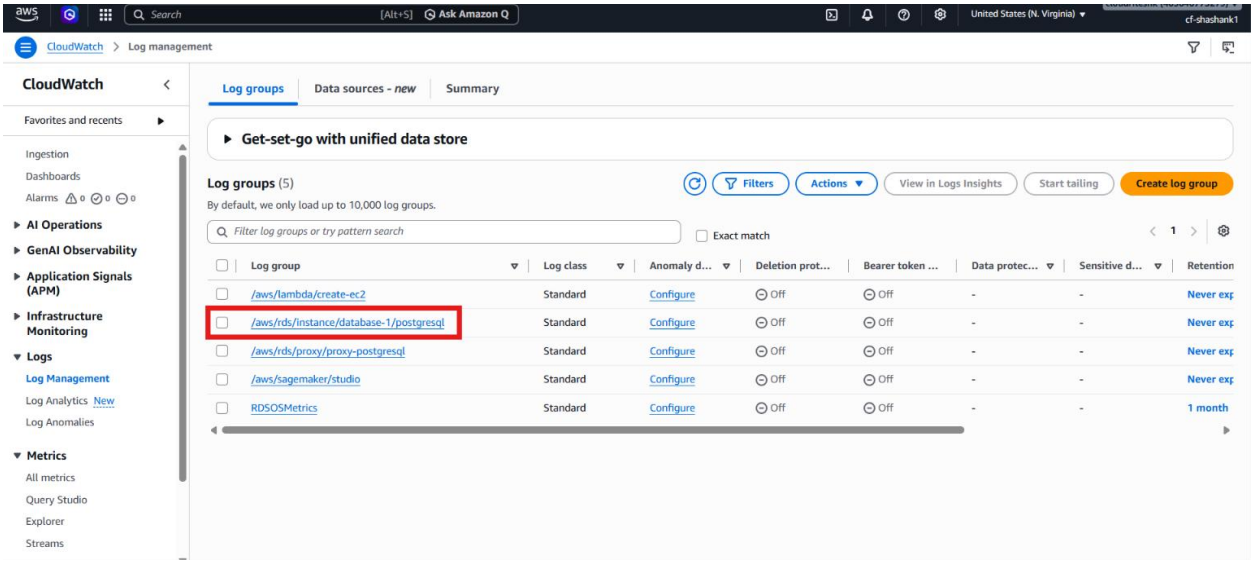
- Now CloudFront will open the page and we can see the streams of our logs being generated.



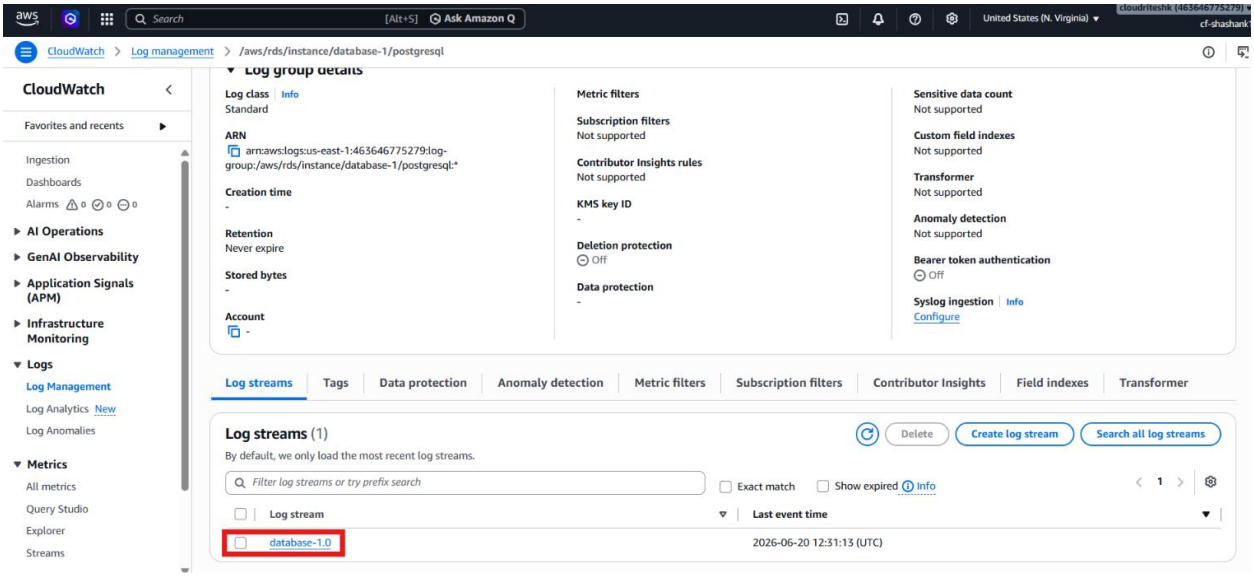
- Scroll down and select one and we can see the query we have ran earlier with prefix **postgres@postgres:[]:LOG:**



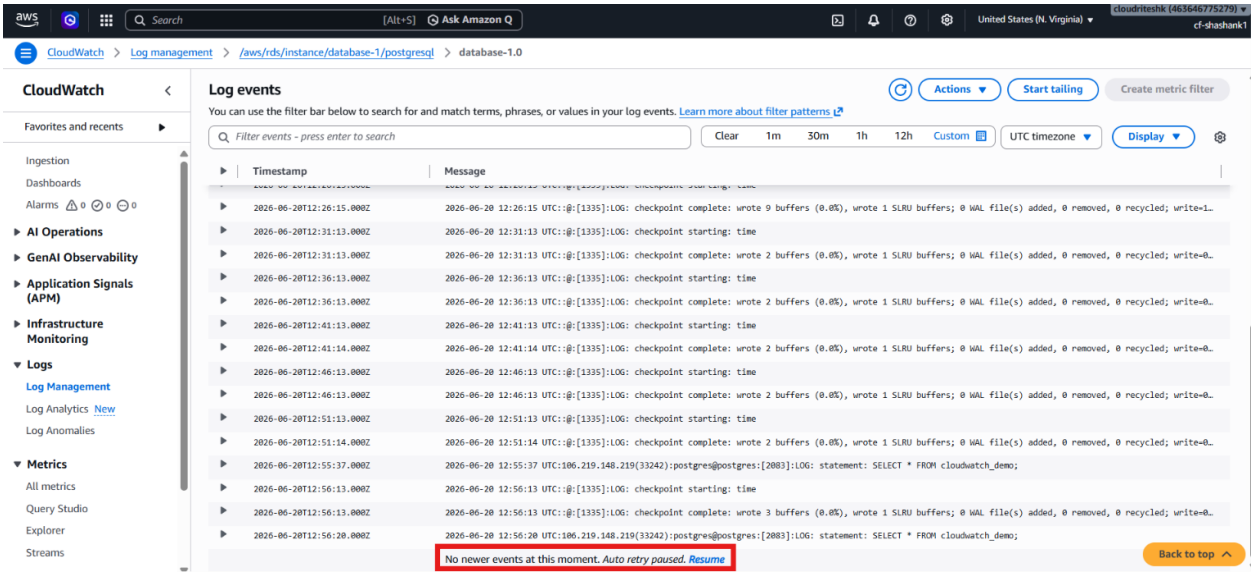
- Here, we can see our CREATE TABLE query we ran earlier.
- To watch it log in real-time, click on Log management on left panel and click on your log group.



- On the **Log Stream** tab, click on your log stream.



- Now click on **Resume** so it will be capturing the logs again.



- Now we can run any statement, and it will get logged.
- We will run this query at the terminal 2 times.

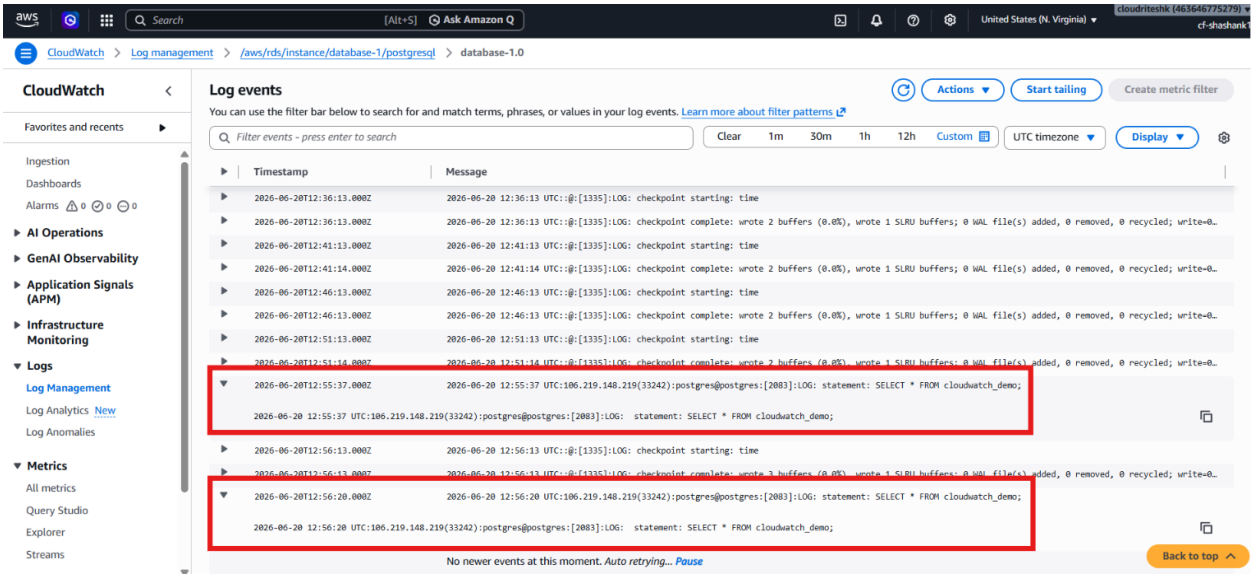
```
SELECT * FROM cloudwatch_demo;
```

```
Command Prompt - psql "ho
postgres(>     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
postgres(> );
CREATE TABLE
postgres=> INSERT INTO cloudwatch_demo
postgres-> (id, customer_name, email, city, amount)
postgres-> VALUES
postgres-> (1,'John Doe','john@example.com','New York',1200.50),
postgres-> (2,'Jane Smith','jane@example.com','Chicago',950.75),
postgres-> (3,'Rahul Verma','rahul@example.com','Delhi',5000.00),
postgres-> (4,'Anita Sharma','anita@example.com','Mumbai',7500.25),
postgres-> (5,'Amit Kumar','amit@example.com','Lucknow',2500.80);
INSERT 0 5
postgres=> SELECT * FROM cloudwatch_demo;
 id | customer_name |       email       |   city    | amount  |         created_at
----+---------------+-------------------+-----------+---------+----------------------------
  1 | John Doe      | john@example.com  | New York  | 1200.50 | 2026-06-20 12:23:02.251274
  2 | Jane Smith    | jane@example.com  | Chicago   |  950.75 | 2026-06-20 12:23:02.251274
  3 | Rahul Verma   | rahul@example.com | Delhi     | 5000.00 | 2026-06-20 12:23:02.251274
  4 | Anita Sharma  | anita@example.com | Mumbai    | 7500.25 | 2026-06-20 12:23:02.251274
  5 | Amit Kumar    | amit@example.com  | Lucknow   | 2500.80 | 2026-06-20 12:23:02.251274
(5 rows)

postgres=> SELECT * FROM cloudwatch_demo;
 id | customer_name |       email       |   city    | amount  |         created_at
----+---------------+-------------------+-----------+---------+----------------------------
  1 | John Doe      | john@example.com  | New York  | 1200.50 | 2026-06-20 12:23:02.251274
  2 | Jane Smith    | jane@example.com  | Chicago   |  950.75 | 2026-06-20 12:23:02.251274
  3 | Rahul Verma   | rahul@example.com | Delhi     | 5000.00 | 2026-06-20 12:23:02.251274
  4 | Anita Sharma  | anita@example.com | Mumbai    | 7500.25 | 2026-06-20 12:23:02.251274
  5 | Amit Kumar    | amit@example.com  | Lucknow   | 2500.80 | 2026-06-20 12:23:02.251274
(5 rows)

postgres=>
```

- And we can see our logs now.


Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
cloudriteshk (463646775279)
cf-shashank
CloudWatch
Log management
/aws/rds/instance/database-1/postgresql
database-1.0
CloudWatch
Favorites and recents
Ingestion
Dashboards
Alarms
AI Operations
GenAI Observability
Application Signals (APM)
Infrastructure Monitoring
Logs
Log Management
Log Analytics New
Log Anomalies
Metrics
All metrics
Query Studio
Explorer
Streams
Log events
You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns
Actions
Start tailing
Create metric filter
Filter events - press enter to search
Clear
1m
30m
1h
12h
Custom
UTC timezone
Display
Timestamp
Message
2026-06-20T12:36:13.000Z
2026-06-20 12:36:13 UTC::@:[1335]:LOG: checkpoint starting: time
2026-06-20T12:36:13.000Z
2026-06-20 12:36:13 UTC::@:[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0…
2026-06-20T12:41:13.000Z
2026-06-20 12:41:13 UTC::@:[1335]:LOG: checkpoint starting: time
2026-06-20T12:41:14.000Z
2026-06-20 12:41:14 UTC::@:[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0…
2026-06-20T12:46:13.000Z
2026-06-20 12:46:13 UTC::@:[1335]:LOG: checkpoint starting: time
2026-06-20T12:46:13.000Z
2026-06-20 12:46:13 UTC::@:[1335]:LOG: checkpoint complete: wrote 2 buffers (0.0%), wrote 1 SLRU buffers; 0 WAL file(s) added, 0 removed, 0 recycled; write=0…
2026-06-20T12:51:13.000Z
2026-06-20 12:51:13 UTC::@:[1335]:LOG: checkpoint starting: time
2026-06-20T12:55:37.000Z
2026-06-20 12:55:37 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;
2026-06-20 12:55:37 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;
2026-06-20T12:56:13.000Z
2026-06-20 12:56:13 UTC::@:[1335]:LOG: checkpoint starting: time
2026-06-20T12:56:20.000Z
2026-06-20 12:56:20 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;
2026-06-20 12:56:20 UTC:106.219.148.219(33242):postgres@postgres:[2083]:LOG: statement: SELECT * FROM cloudwatch_demo;
No newer events at this moment. Auto retrying... Pause
Back to top